

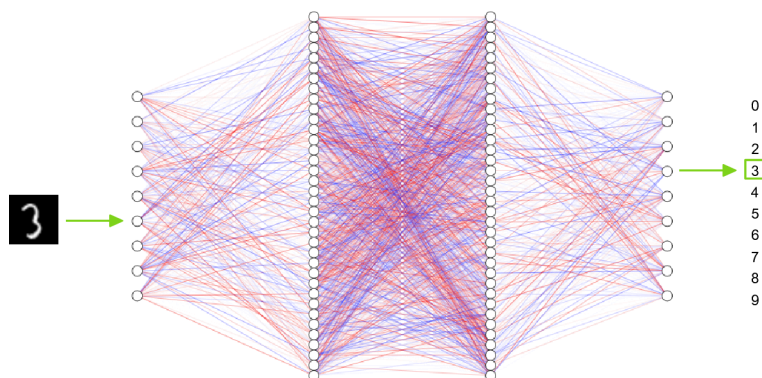
République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
UNIVERSITE MOULOUD MAMMERI DE TIZI-OUZOU
FACULTE DU GENIE ELECTRIQUE ET D' INFORMATIQUE
DEPARTEMENT D'INFORMATIQUE



TP2

Thème

Reconnaissance de caractères avec KNN et CNN



Réalisé par :	Responsable du module :
- REBHI Idir - LARBI Siham	M.MohammedOurabah SOUALAH

Table des matières :

1.Introduction.....	5
2.Position du problème.....	5
2.1. Les défis techniques.....	5
2.2. Architecture globale.....	5
L'approche CNN :	5
2.3. Analyse des Structures de Données.....	6
3. k-Nearest Neighbors (KNN).....	6
4. Convolutional Neural Networks (CNN).....	7
4. Analyse et Conception de la Solution.....	7
1. L'Approche par K-Nearest Neighbors (KNN).....	9
A. Prétraitement et Segmentation (Algorithme de Projection).....	9
B. Normalisation et Réduction de Dimension (PCA).....	9
C. Classification par Distance.....	9
D. Évaluation des performances.....	10
E.Analyse des erreurs (Matrice de confusion).....	10
F.Interface de démonstration (Streamlit).....	11
2. L'Approche par Réseaux de Neurones Convolutifs (CNN).....	12
A. Architecture du Réseau.....	12
B. Avantages de la Conception.....	13
C. Processus d'Entraînement et Optimisation du CNN.....	13
1.Résultats du modèle KNN.....	16
1.1.Limites du système.....	16
2.Résultats du modèle CNN.....	16
Le CNN a été évalué sur deux plans : les données standards (MNIST) et un test "maison" pour éprouver sa robustesse.....	16
2.1 Performances sur MNIST.....	16
2.2 Test en conditions réelles :	17
2.3 Analyse de l'erreur (Le "1" prédit en "4").....	18
3.Tableau comparatif KNN vs CNN.....	18
Une perspective intéressante pour la segmentation :	19
Limite de ta méthode actuelle.....	19
4.Conclusion générale.....	21

I. Introduction

1.Introduction

La reconnaissance de caractères manuscrits est un défi majeur en vision par ordinateur, notamment pour la numérisation automatique de documents (formulaires, chèques, courriers). Le problème consiste à transformer une information visuelle brute (pixels d'une image scannée) en une information textuelle exploitable par une machine.

Dans ce projet, nous devons traiter des images de chiffres alignés, extraire chaque entité par segmentation, puis les identifier en utilisant des modèles d'apprentissage automatique entraînés sur la base de données de référence MNIST

2.Position du problème

La reconnaissance automatique de caractères (OCR - *Optical Character Recognition*) est un domaine fondamental de la vision par ordinateur. Le défi consiste à transformer une information visuelle non structurée (une matrice de pixels représentant des chiffres manuscrits) en une information textuelle structurée (des entiers ...etc).

2.1. Les défis techniques

Le problème se décompose en trois sous-problèmes majeurs que nous avons dû résoudre :

1. **L'isolation (Segmentation)** : Comment, dans une image contenant "123", séparer proprement le "1", le "2" et le "3" ?
2. **La variabilité morphologique** : Deux personnes n'écrivent jamais un "4" de la même manière. Le système doit être capable de généraliser.
3. **La dimensionnalité** : Une image de 28x28 pixels représente 784 variables. Traiter autant de données demande des structures de données optimisées et des algorithmes de réduction de dimension.

2.2. Architecture globale

Le système suit le flux de données suivant :

Image Brute → Binarisation → Projection Verticale → Normalisation → Classification (KNN ou CNN) .

Nous avons exploré deux paradigmes de classification :

L'approche CNN :

Bien que le TP se concentrait sur le KNN, nous avons pris l'initiative d'implémenter un modèle de **Deep Learning (CNN)** pour pallier les faiblesses du KNN et aussi dans le but de comparer un peu les deux méthodes

- **L'approche géométrique (KNN) :** Fondée sur la proximité dans l'espace euclidien. Elle repose sur l'hypothèse que des images similaires (au sens de la distance) appartiennent à la même classe.
- **L'approche par extraction de traits hiérarchiques (CNN) :** Contrairement au KNN, le Réseau de Neurones Convolutifs apprend ses propres filtres pour détecter des formes (bords, boucles, angles) de manière invariante par rapport à la position.

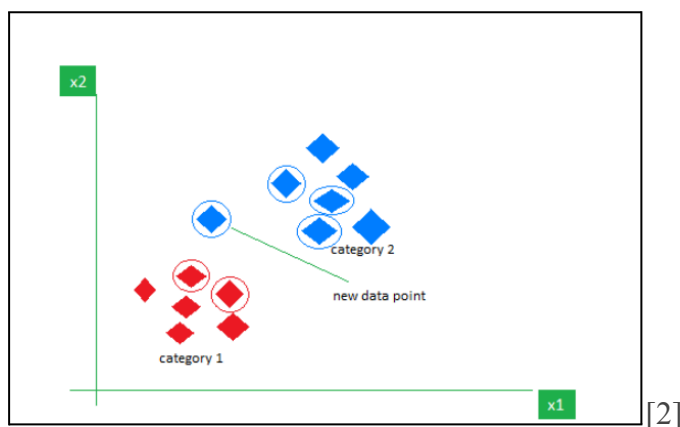
2.3. Analyse des Structures de Données

Nous utilisons principalement trois structures de données pour manipuler l'information :

- **Les Tableaux Multidimensionnels (NumPy Arrays) :** Pour représenter les images sous forme de matrices de pixels $I(x, y)$
- **Vecteurs de caractéristiques :** Pour le modèle KNN, l'image 2D est aplatie en un vecteur 1D de 784 éléments.
- **Tenseurs (4D) :** Pour le modèle CNN, les données sont structurées en $(N, 28, 28, 1)$, où N est le nombre d'images et 1 le canal de couleur (gris).

3. k-Nearest Neighbors (KNN)

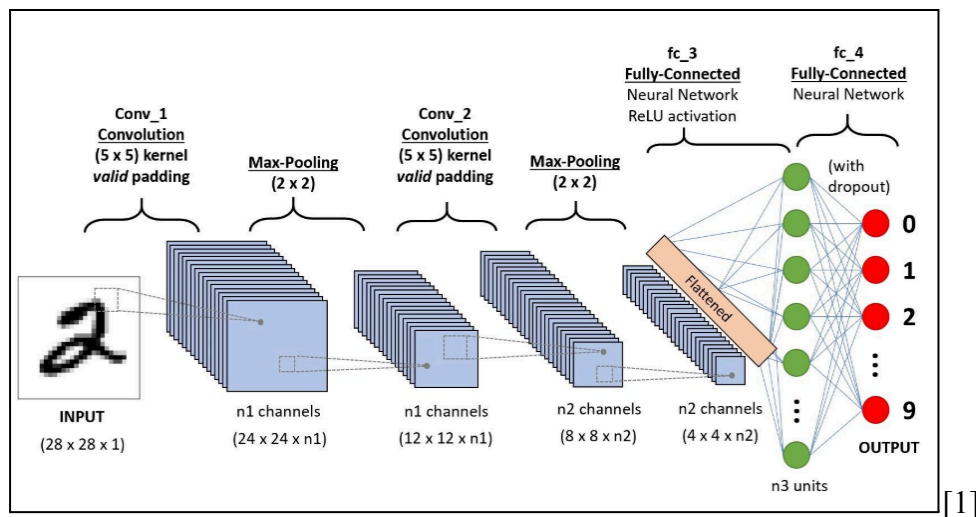
Les **KNN (k-Nearest Neighbors)** sont des algorithmes d'apprentissage supervisé basés sur l'instance, utilisés aussi bien en classification qu'en régression. Le principe est simple : pour prédire la classe d'un nouvel échantillon, l'algorithme calcule sa distance (souvent euclidienne) avec tous les exemples du jeu d'entraînement, sélectionne les k plus proches voisins, puis attribue la classe majoritaire parmi ces voisins. Les KNN ne nécessitent pas de phase d'entraînement explicite (on parle d'apprentissage « paresseux »), mais ils peuvent devenir coûteux en temps de calcul lors de la phase de prédiction, surtout lorsque le jeu de données est volumineux. De plus, leurs performances dépendent fortement du choix de k , de la métrique de distance et de la normalisation des données.



KNN

4. Convolutional Neural Networks (CNN)

Les **CNN (Convolutional Neural Networks)** sont un type de réseau de neurones artificiels spécialement conçu pour traiter des données structurées en grille, comme les images. Contrairement aux KNN, les CNN apprennent automatiquement des représentations hiérarchiques grâce à des couches de convolution, d'activation et de pooling. Les filtres convolutifs permettent d'extraire des caractéristiques locales (bords, textures, formes), qui deviennent de plus en plus abstraites dans les couches profondes. Les CNN nécessitent une phase d'entraînement intensive avec optimisation par rétropropagation et descente de gradient, mais ils sont extrêmement performants pour des tâches complexes comme la reconnaissance d'images ou la stéganalyse. En résumé, les KNN sont simples, interprétables et adaptés à de petits jeux de données structurés, tandis que les CNN sont plus puissants et adaptés aux données de grande dimension, notamment visuelles, au prix d'une complexité computationnelle bien plus élevée.



La figure représente une architecture CNN avec deux couches de neurones fully-connected vers la fin

4. Analyse et Conception de la Solution

Pour répondre au problème, nous avons conçu un pipeline de traitement divisé en quatre phases distinctes :

1. **Prétraitement** : Binarisation de l'image pour séparer le texte (premier plan) du fond.
2. **Segmentation** : Identification des limites de chaque caractère pour isoler les chiffres.
3. **Normalisation** : Redimensionnement et centrage des segments pour les rendre compatibles avec le format MNIST (28×28 pixels).
4. **Classification** : Utilisation d'algorithmes (KNN , CNN) pour prédire la valeur du chiffre

II. Conception et implémentation

Cette partie décrit la structure technique de notre solution, divisée en deux approches distinctes.

1. L'Approche par K-Nearest Neighbors (KNN)

L'approche KNN constitue la base de notre étude. Elle repose sur la comparaison directe de la morphologie des chiffres.

A. Prétraitement et Segmentation (Algorithme de Projection)

Avant la classification, nous devons isoler chaque chiffre. Nous avons conçu un algorithme de **projection verticale** :

- **Logique** : On calcule un histogramme $V(x)$ représentant la somme des intensités des pixels pour chaque colonne x .
- **Algorithme** : Les zones où $V(x) > 0$ indiquent la présence de matière (pixel blanc du caractère). Les zones où $V(x) = 0$ sont les séparateurs
- **Structure de données** : Nous utilisons des matrices **NumPy** pour manipuler ces segments et les isoler dans des "Bounding Boxes".

B. Normalisation et Réduction de Dimension (PCA)

Chaque segment est redimensionné en 28x28 pixels et centré pour correspondre au format du dataset **MNIST**.

Cependant, travailler dans un espace de 784 dimensions ralentit le KNN. Nous avons donc implémenté une **Analyse en Composantes Principales (PCA)**.

- **Conception** : La PCA réduit le vecteur de 784 dimensions à un vecteur réduit (environ 50 composantes) en ne gardant que les axes de variance maximale, ce qui accélère la recherche des voisins sans perdre l'essence du tracé.

C. Classification par Distance

La classification est réalisée à l'aide de l'algorithme **K-Nearest Neighbors (KNN)**. Le principe consiste à calculer la **distance euclidienne** entre le vecteur représentant un nouveau caractère et l'ensemble des vecteurs du jeu d'apprentissage. Les k voisins les plus proches sont ensuite sélectionnés, et la classe majoritaire parmi ces voisins est attribuée au caractère étudié. Dans notre cas, nous avons fixé le paramètre $k = 3$, conformément aux recommandations du TP.

L'utilisation de la **distance euclidienne** permet de mesurer la **similarité géométrique** entre deux images de caractères manuscrits. Plus la distance entre deux vecteurs est faible, plus les caractères sont considérés comme similaires

D. Évaluation des performances

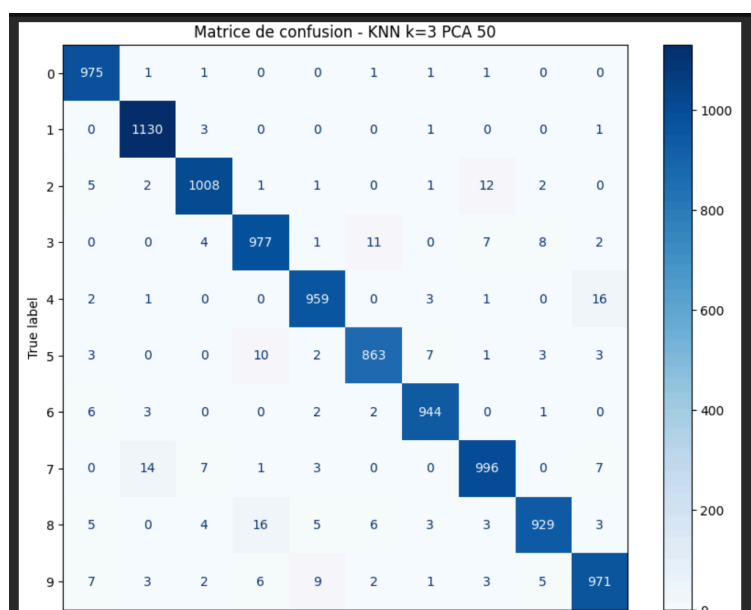
Le modèle a été évalué à l'aide du jeu de données **MNIST**, séparé en un ensemble d'apprentissage et un ensemble de test. Les images du dataset ont été normalisées dans l'intervalle $[0,1]$ avant l'entraînement du modèle.

La performance du système est mesurée à l'aide de la précision (**accuracy**), qui représente le pourcentage de caractères correctement classés.

Les résultats expérimentaux montrent une précision d'environ **97.52%**, ce qui confirme l'efficacité du pipeline proposé pour la reconnaissance de chiffres manuscrits. L'application de la PCA permet également de réduire le temps de traitement sans dégrader significativement la précision du modèle.

E. Analyse des erreurs (Matrice de confusion)

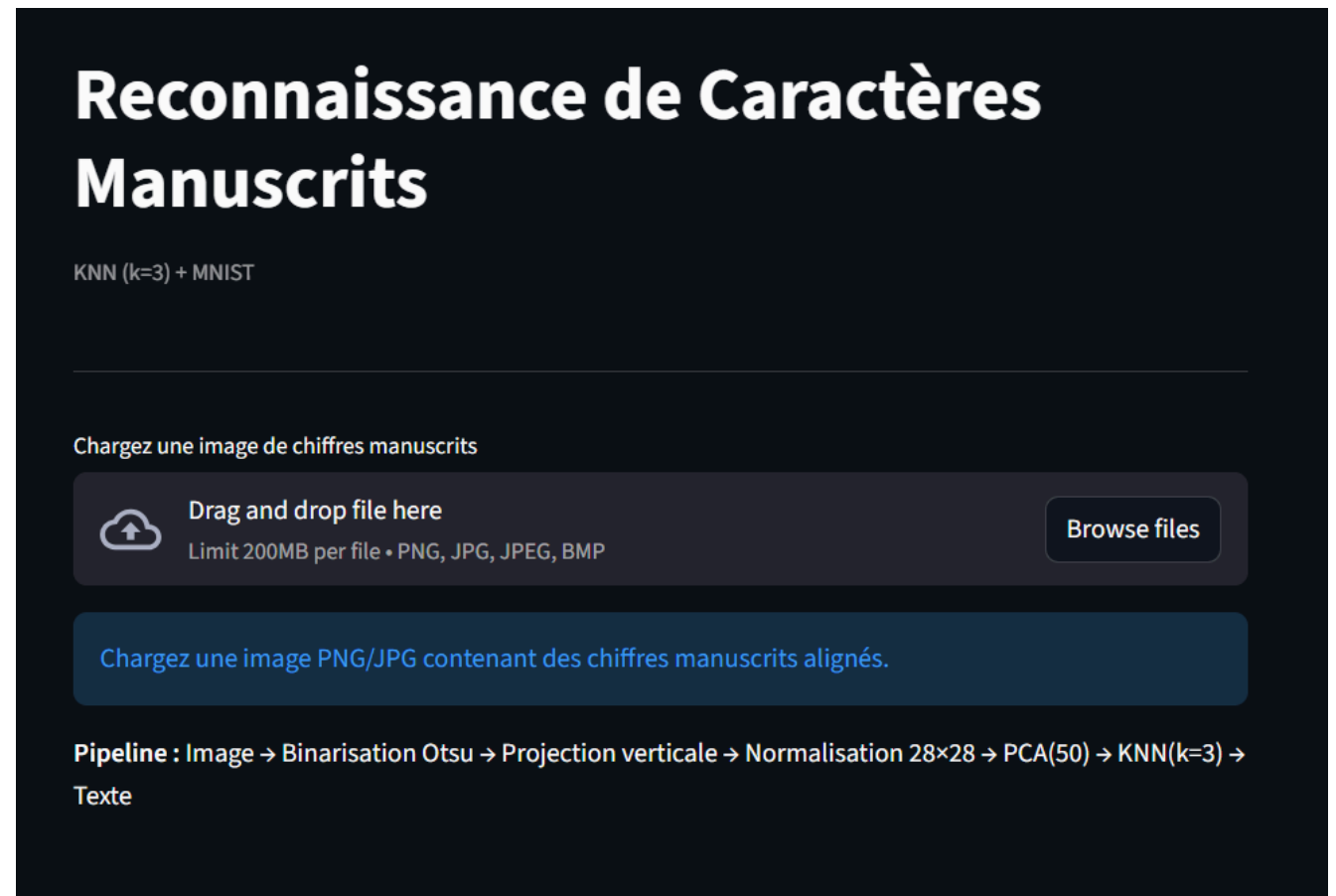
Pour analyser plus finement les performances du modèle, nous avons calculé la matrice de confusion sur le jeu de test. Chaque ligne représente la classe réelle et chaque colonne la classe prédite. Les valeurs sur la diagonale correspondent aux prédictions correctes. On observe que les confusions les plus fréquentes surviennent entre des chiffres morphologiquement proches, ce qui s'explique par leur similarité visuelle pixel à pixel. Cette analyse confirme que le KNN reste performant mais reste sensible à la forme brute des caractères.



F. Interface de démonstration (Streamlit)

Afin de rendre le système accessible et démontrable, nous avons développé une interface interactive avec **Streamlit**. L'utilisateur peut charger une image contenant des chiffres manuscrits alignés. Le système effectue automatiquement la segmentation, la normalisation, puis la prédiction de chaque caractère. Les segments normalisés en 28×28 sont affichés avec leur chiffre prédit en dessous, et le résultat final est reconstruit sous forme de chaîne de caractères. Les modèles sont chargés une seule fois en mémoire grâce à [@st.cache_resource](#), ce qui garantit une réponse rapide.

Cette interface permet de valider visuellement le bon fonctionnement du système de reconnaissance et facilite la démonstration du projet lors de la présentation.



interface streamlit

2. L'Approche par Réseaux de Neurones Convolutifs (CNN)

Pour pallier les limites du KNN (sensibilité aux décalages de quelques pixels), nous avons implémenté un modèle de **Deep Learning**.

A. Architecture du Réseau

Contrairement au KNN, le CNN n'utilise pas de vecteurs plats mais des **Tenseurs 4D** (batch, largeur, hauteur, canaux). Notre architecture comprend :

1. **Couches de Convolution** : Application de filtres 3x3 pour extraire des traits élémentaires (horizontaux, courbes).
2. **Couches de Pooling (Max Pooling)** : Réduction de la taille spatiale pour rendre le modèle invariant aux petites translations du chiffre dans son cadre.
3. **Couches Denses (Fully Connected)** : Une couche finale "Softmax" qui produit une distribution de probabilité sur les 10 chiffres possibles.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 8)	80
max_pooling2d (MaxPooling2D)	(None, 13, 13, 8)	0
dropout (Dropout)	(None, 13, 13, 8)	0
conv2d_1 (Conv2D)	(None, 11, 11, 16)	1,168
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 16)	0
dropout_1 (Dropout)	(None, 5, 5, 16)	0
flatten (Flatten)	(None, 400)	0
dense (Dense)	(None, 100)	40,100
dropout_2 (Dropout)	(None, 100)	0
dense_1 (Dense)	(None, 10)	1,010

Total params: 42,358 (165.46 KB)
Trainable params: 42,358 (165.46 KB)
Non-trainable params: 0 (0.00 B)

Cette figure montre un summary de notre réseau CNN

```
#Construction de notre modèle

model = keras.models.Sequential()

model.add(keras.layers.Input((28,28,1)))

model.add(keras.layers.Conv2D(8, (3,3), activation='relu'))
model.add(keras.layers.MaxPooling2D((2,2)))
model.add(keras.layers.Dropout(0.2))

model.add(keras.layers.Conv2D(16, (3,3), activation='relu'))
model.add(keras.layers.MaxPooling2D((2,2)))
model.add(keras.layers.Dropout(0.2))

#Mettre tout à plat
model.add(keras.layers.Flatten())
model.add(keras.layers.Dense(100, activation='relu'))
model.add(keras.layers.Dropout(0.5))

model.add(keras.layers.Dense(10, activation='softmax'))
```

Construction du modèle avec le modèle sequential de Keras

NB : Les paramètres comme le nombre de couches à mettre est définie expérimentalement, on pourrait éventuellement jouer avec ce genre de paramètres, par exemple 16 couches CNN au départ

B. Avantages de la Conception

Alors que le KNN est un modèle "paresseux" qui nécessite de stocker toutes les données, le CNN apprend des **poids optimisés**. Une fois entraîné, le modèle est extrêmement léger et beaucoup plus robuste aux déformations et aux bruits que l'approche KNN.

C. Processus d'Entraînement et Optimisation du CNN

L'efficacité d'un réseau de neurones ne dépend pas seulement de son architecture, mais de la rigueur de son protocole d'apprentissage. Nous avons utilisé la base de données **MNIST** (60 000 images d'entraînement) avec le paramétrage suivant :

- **Fonction de Perte (Loss Function) :** Nous avons choisi la **Categorical Cross-Entropy**. Étant donné que nous sommes face à un problème de classification multi-classe (10 chiffres), cette fonction mesure l'écart entre la distribution de probabilité prédite par le modèle (via Softmax) et la réalité
- **Optimiseur :** L'algorithme **Adam** (*Adaptive Moment Estimation*) a été privilégié. Contrairement à une descente de gradient classique, Adam ajuste dynamiquement le taux d'apprentissage pour chaque paramètre, ce qui permet une convergence plus rapide et évite de rester bloqué dans des minima locaux.
- **Hyperparamètres :**
 - **Batch Size (Taille de lot) :** Fixé à 512

- **Époques** : Le modèle a été entraîné sur 16 epochs complètes, ce qui suffit pour atteindre une bonne précision sur MNIST sans risquer le surapprentissage (*overfitting*), si on augmente le nombre d'epochs on risque le surapprentissage donc on aura une probable chute d'accuracy
- **Validation Split** : Nous avons réservé 10% des données d'entraînement pour la validation. Cela nous permet de surveiller en temps réel si le modèle généralise bien sur des données qu'il n'a pas "vues" pendant la mise à jour des poids

```
#Entraînement

batch_size = 512
epochs = 16

history = model.fit(x_train, y_train,
                    batch_size = batch_size,
                    epochs = epochs,
                    verbose = 1,
                    validation_data = (x_test, y_test))
```

III. Résultats et conclusion

1. Résultats du modèle KNN

Le modèle KNN (k=3, distance euclidienne, PCA 50 composantes) a atteint une précision de **97.52%** sur les 10 000 images de test MNIST. La variance conservée par la PCA est d'environ **82.5%** .

```
# entraînement du modele knn
knn = KNeighborsClassifier(n_neighbors=3, metric='euclidean', n_jobs=-1)
knn.fit(X_train_pca, y_train)
acc = accuracy_score(y_test, knn.predict(X_test_pca))
print(f"Precision KNN : {acc*100:.2f}%")

with open('knn.pkl', 'wb') as f: pickle.dump(knn, f)
with open('pca.pkl', 'wb') as f: pickle.dump(pca, f)
print("Modeles sauvegardes OK")
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 ————— 2s 0us/step
Variance PCA : 82.5%
Precision KNN : 97.52%
Modeles sauvegardes OK

1.1. Limites du système

Malgré de bons résultats, le système présente certaines limites.

L'algorithme KNN peut devenir coûteux en temps de calcul lorsque le nombre d'exemples d'apprentissage est très grand. De plus, la méthode est sensible au bruit, à la position des caractères et à la variation des styles d'écriture.

Le système fonctionne principalement sur des images propres et bien alignées. La reconnaissance d'écritures complexes ou cursives peut réduire la précision du modèle.

L'utilisation de modèles d'apprentissage profond, comme les **réseaux de neurones convolutifs (CNN)**, pourrait améliorer la robustesse et la performance du système

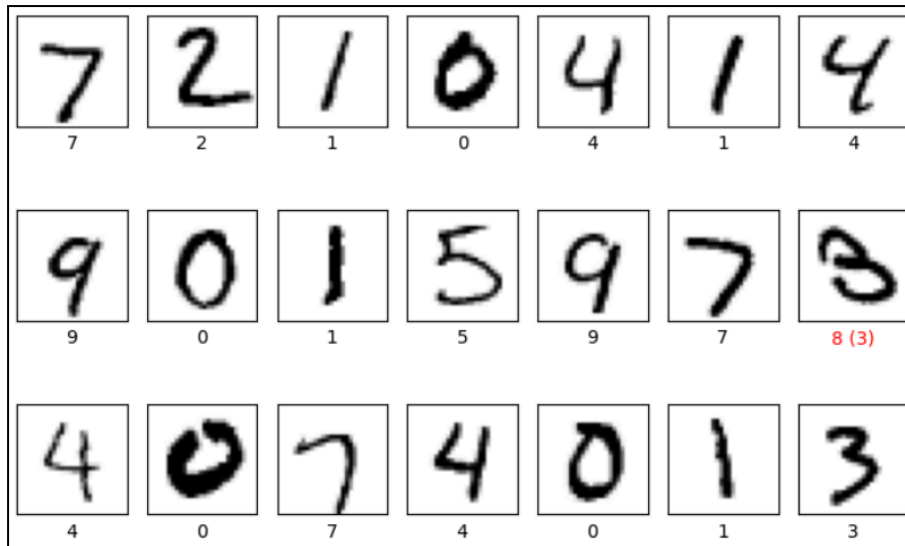
2. Résultats du modèle CNN

Le CNN a été évalué sur deux plans : les données standards (MNIST) et un test "maison" pour éprouver sa robustesse.

2.1 Performances sur MNIST

Le modèle a atteint une précision de **98.8%** sur les données de tests

III. Résultats et conclusion

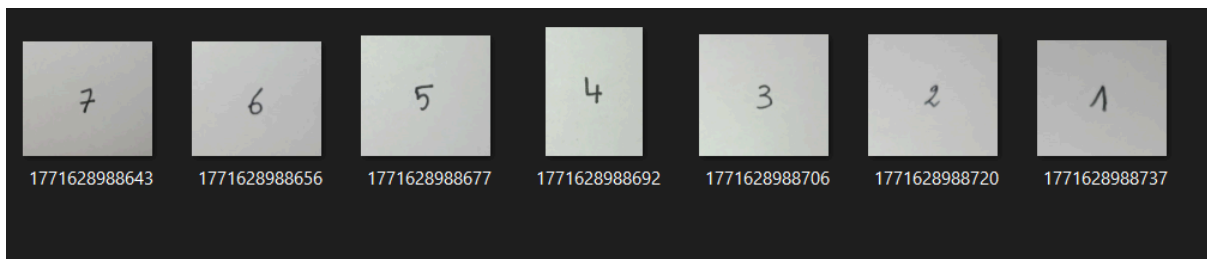


Cette image représente quelques prédictions faites par notre modèle CNN

→ On voit que le modèle a prédit un '3' comme étant un '8', il a été conduit à l'erreur car ce 3 en particulier ressemble fortement à un '8', si le modèle s'entraîne sur plus de données il deviendra plus robuste face à ce genre d'écritures

2.2 Test en conditions réelles :

Nous avons testé le modèle sur nos propres écritures.

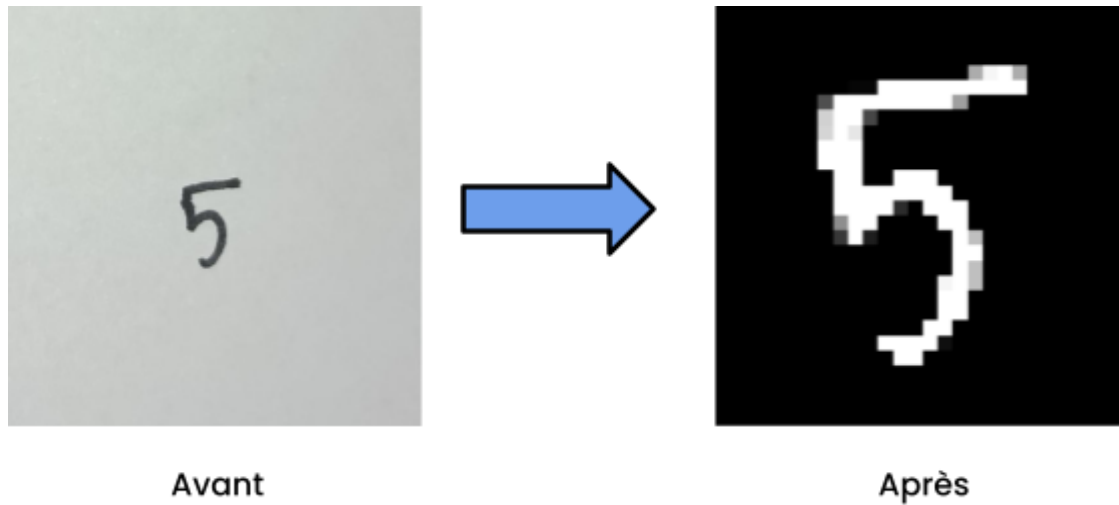


Images des nos propres écritures

- **Protocole :** Écriture manuscrite des chiffres de 1 à 7 sur papier blanc, capture photo via smartphone, transfert et traitement Python.
- **Pipeline de prétraitement critique :** Pour que l'image soit "compréhensible" par le modèle, nous avons implémenté un pipeline de prétraitement (il faut que les images soient comme celle de MNIST, le modèle n'a pas une aussi grande capacité de généralisation car il a été entraîné sur peu de données, si les images de tests sont un peu différentes de MNIST il aura du mal à les classifier) :
 1. **Réduction de bruit :** Flou Gaussien (5,5)
 2. **Seuillage d'Otsu :** Binarisation adaptative pour isoler le chiffre malgré le mauvais éclairage de la photo
 3. **Extraction de contours :** Utilisation de [cv2.findContours](#) pour localiser précisément le chiffre et éliminer les bords de la feuille.
 4. **Normalisation :** Redimensionnement gardant le ratio d'aspect (tenu dans un cadre de 20x20pixels) puis insertion dans une matrice 28x28 avec **padding**.

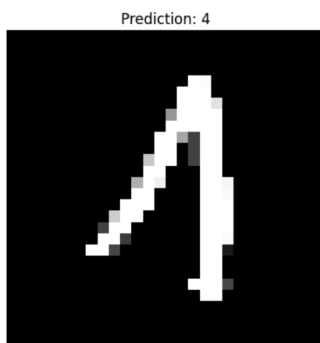
III. Résultats et conclusion

Résultats du prétraitement :



2.3 Analyse de l'erreur (Le "1" prédit en "4")

Sur les 7 chiffres testés, le CNN a réalisé un score de **6/7 (85.7% de réussite)**. L'unique erreur est la prédiction d'un 4 à la place d'un 1.



Accéder à l'implémentation KNN	TP2_KNN_VISION_ARTIFICIELLE.ip...
Lien streamlit	https://unattempted-unsieged-kasie.ngrok-free.dev
Accéder à l'implémentation CNN	ClassificationDesCarAvecCNN.ipynb

À noter que si le lien Streamlit ne marche pas il suffit de ré exécuter l'implémentation KNN de la première à la dernière cellule et ça va générer automatiquement un nouveau lien, mais attention il faut ignorer la cellule suivante qui sert à fermer l'application :

```
5. Arrêter l'application
```

```
proc.terminate()
ngrok.disconnect(public_url)
ngrok.kill()
print("Application arretee.")
```

Cellule à ignorer

III. Résultats et conclusion

→ On vous propose une image pour tester le KNN, voici le lien pour la télécharger, comme vous pouvez tester avec n'importe quelle autre image

Lien :  1000021497.jpg

3. Tableau comparatif KNN vs CNN

Critère	KNN	CNN
Précision MNSIT	97.52%	98.8%
Robustesse réelle	Moyenne	Bonne
Temps de prédiction	Lent	Rapide
Complexité	Simple	Elevée

NB = la solution basée sur les CNN n'inclut pas une segmentation d'images, il est important de la tester uniquement avec des chiffres isolés

Autre remarque : La démonstration avec streamlit ne concerne que la solution avec KNN aussi, Cette solution n'est là qu'à titre de comparaison uniquement

Une perspective intéressante pour la segmentation :

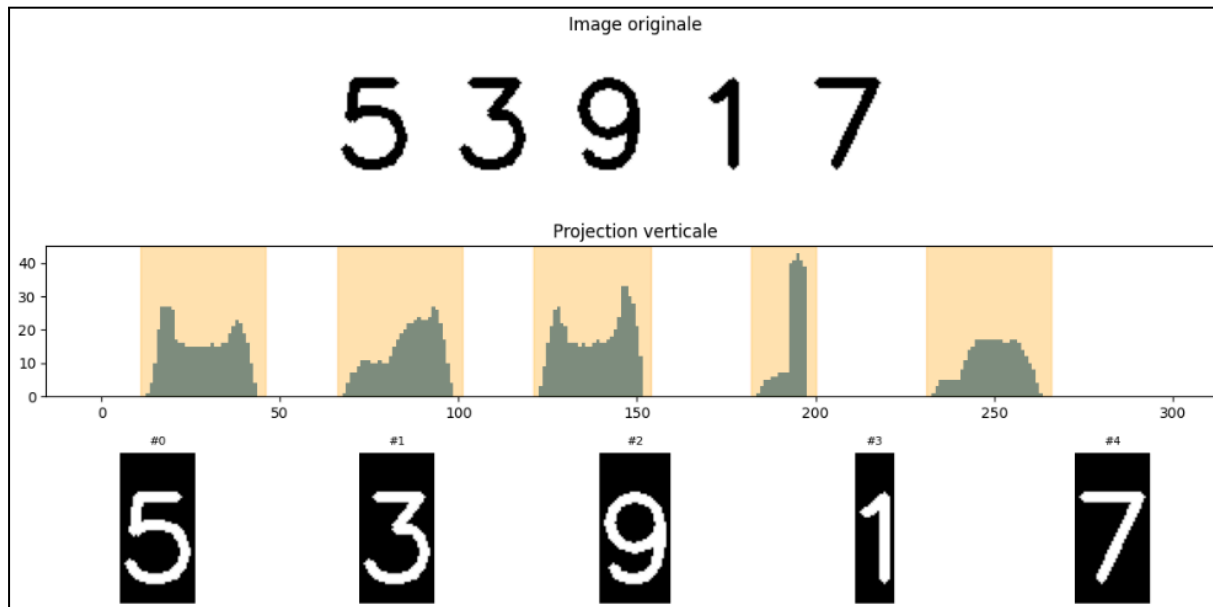
Limite de ta méthode actuelle

La **projection verticale de l'histogramme** fonctionne bien seulement si :

- les objets sont **bien séparés**
- les espacements sont **réguliers**
- il n'y a **pas trop de bruit**
- les objets sont **alignés verticalement**

III. Résultats et conclusion

Exemple d'une bonne segmentation (car toutes les conditions sont réunies vu que c'est des données synthétiques sur lesquels on a le contrôle :

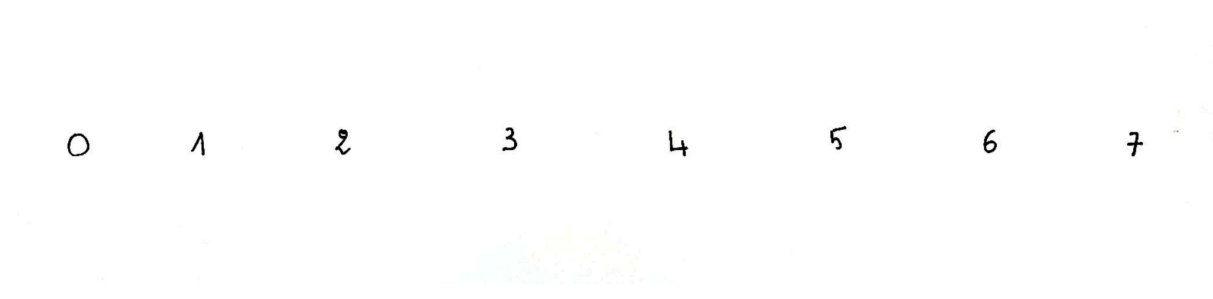


Mais elle échoue quand :

- les objets **se touchent**
- l'image est **bruitée**
- les formes sont **irrégulières**
- il y a **des rotations ou déformations**

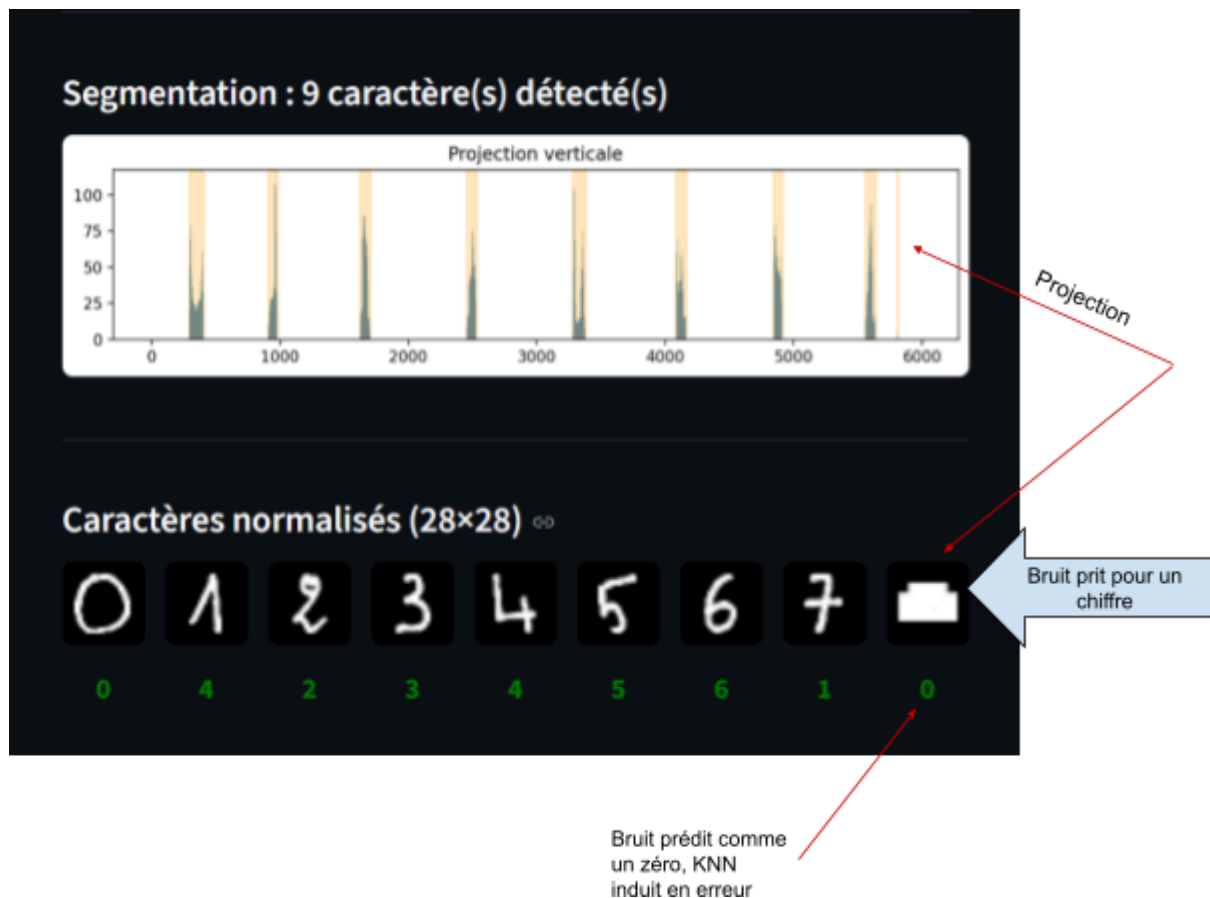
Exemple d'une segmentation induite en erreur à cause d'un bruit pris pour un chiffre (cet exemple est issu d'une autre image des chiffres qu'on a écrit à la main avec laquelle on a testé le KNN) :

L'image :



III. Résultats et conclusion

Résultat de la segmentation :

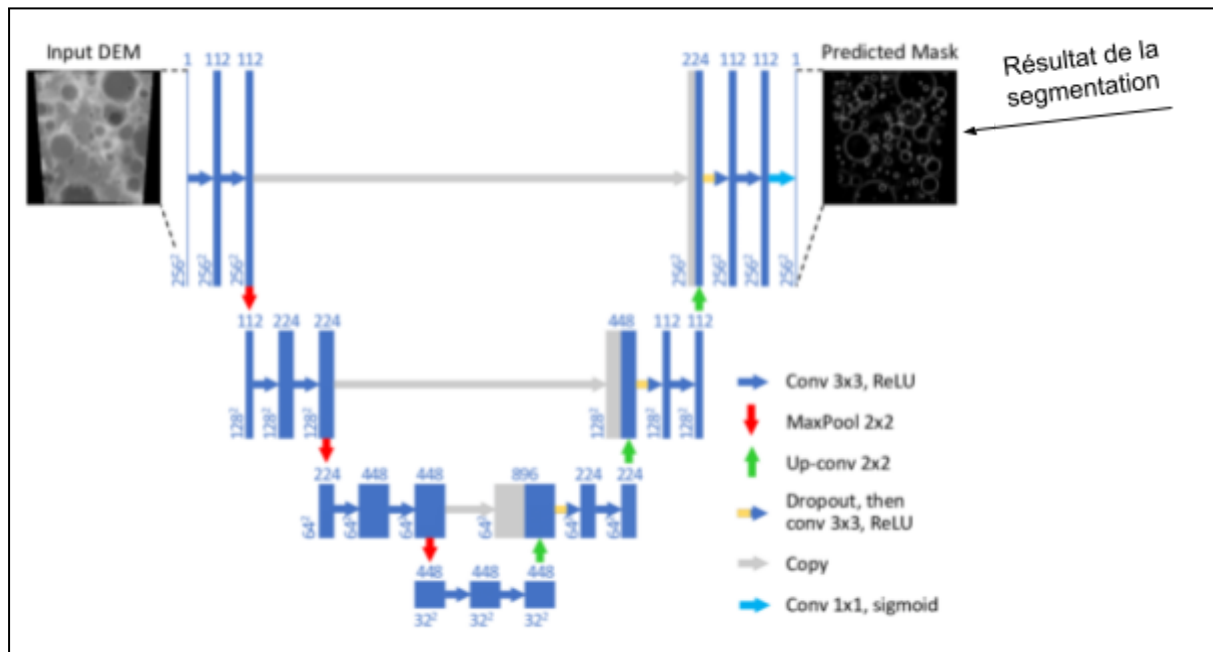


Solution :

Pour améliorer le système de segmentation consisterait à remplacer l'approche actuelle basée sur la projection verticale de l'histogramme par une méthode de segmentation reposant sur des architectures de type **U-Net**. En effet, la segmentation par projection verticale, bien que simple et rapide à mettre en œuvre, reste sensible au bruit, aux variations d'espacement et aux irrégularités présentes dans les données. À l'inverse, un modèle U-Net, largement utilisé dans les tâches de segmentation d'images, permet d'exploiter l'information spatiale complète de l'image grâce à son architecture encodeur-décodeur et à ses connexions de saut (skip connections). Cette approche pourrait permettre une segmentation plus robuste et plus précise, notamment dans les cas où les structures à détecter sont irrégulières ou partiellement dégradées. L'intégration d'un tel modèle constituerait ainsi une piste prometteuse pour améliorer la qualité globale de la segmentation.

III. Résultats et conclusion

Exemple d'une architecture U-NET :



[3]

4. Conclusion générale

Ce projet a permis de concevoir et de mettre en œuvre un système complet de reconnaissance de caractères manuscrits basé sur des techniques de vision par ordinateur et d'apprentissage automatique. L'objectif principal était d'analyser et de comparer deux approches de classification, à savoir l'**algorithme K-Nearest Neighbors (KNN)** combiné à la réduction de dimension par **PCA**, ainsi qu'un modèle de réseaux de neurones convolutifs (CNN).

Les résultats expérimentaux obtenus montrent l'efficacité du pipeline proposé, incluant la segmentation des caractères, la normalisation au format MNIST, l'extraction de caractéristiques et la classification. Le modèle KNN a atteint une précision satisfaisante d'environ **97.52%** sur le jeu de test MNIST, tandis que le modèle CNN a obtenu une précision encore supérieure de **98.8%**, confirmant la supériorité des approches d'apprentissage profond pour les tâches de reconnaissance d'images.

Cependant, chaque méthode présente ses propres limites. Le **KNN** reste sensible aux variations d'écriture, au bruit et aux translations légères des caractères, tandis que le **CNN** nécessite un temps d'entraînement plus important et dépend fortement de la quantité et de la diversité des données d'apprentissage.

III. Résultats et conclusion

Ce projet a également permis de développer une interface interactive avec Streamlit, facilitant la démonstration du système en conditions réelles. Cette réalisation constitue une bonne base pour l'étude des systèmes **OCR** et des techniques modernes de reconnaissance d'images.

En perspective, plusieurs améliorations peuvent être envisagées, notamment l'utilisation d'architectures de réseaux de neurones plus avancées, l'augmentation artificielle des données d'apprentissage ainsi que l'adaptation du système à la reconnaissance d'écritures plus complexes ou cursives.

IV. Références :

1. **Ratan, P.** (2025, 1 mai). *What is the Convolutional Neural Network Architecture?* Analytics Vidhya.
<https://www.analyticsvidhya.com/blog/2020/10/what-is-the-convolutional-neural-net-work-architecture/>
2. **GeeksforGeeks.** (2025, 2 mai). *KNN Classifier in R Programming.* GeeksforGeeks.
<https://www.geeksforgeeks.org/r-language/k-nn-classifier-in-r-programming/>
3. Oxbowerce. (2021, May 1). *Why do we need to concatenate in a U-Net?* Data Science Stack Exchange.
<https://datascience.stackexchange.com/questions/93805/why-do-we-need-to-concatenate-in-a-u-net>